TUẦN 1 – BUỔI 1

1. CÁC BƯỚC CÀI Code::Block:

Bước 1: Tải bộ cài đặt phù hợp về máy

1. Em mở trình duyệt mạng trên máy tính và truy cập vào trang tải chính thức của phần mềm: codeblocks.org/downloads/binaries.
2. Em kéo chuột xuống phần dành cho hệ điều hành Windows.
3. **Lưu ý quan trọng nhất:** Em hãy tìm đúng dòng tệp có tên là **codeblocks-20.03mingw-setup.exe**.
    - *Lý do:* Tệp có chữ mingw đã đi kèm sẵn bộ dịch mã nguồn sang ngôn ngữ máy. Nếu tải bản không có chữ này, chương trình sẽ không dịch và chạy được mã lệnh.
4. Em bấm vào liên kết tải từ máy chủ Fosshub hoặc Sourceforge rồi chờ tệp tải về máy tính.

Bước 2: Tiến hành cài đặt phần mềm

1. Em mở thư mục vừa tải về, nhấp đúp chuột trái vào tệp codeblocks-20.03mingw-setup.exe.
2. Hộp thoại cài đặt xuất hiện, em nhấn **Next** để bắt đầu.
3. Ở màn hình điều khoản sử dụng, em nhấn chọn **I Agree**.
4. Ở màn hình lựa chọn thành phần cài đặt, em giữ nguyên toàn bộ các dấu tích mặc định rồi nhấn **Next**.
5. Em chọn thư mục lưu phần mềm (mặc định là ổ đĩa C) rồi nhấn nút **Install**. Quá trình giải nén và sao chép tệp sẽ diễn ra trong khoảng một đến hai phút.
6. Khi hoàn tất, một bảng thông báo nhỏ hỏi *"Do you want to run Code::Blocks now?"*, em có thể nhấn **Yes** để mở ngay phần mềm, hoặc nhấn **No** rồi bấm **Next** và **Finish** để đóng trình cài đặt.

Bước 3: Kiểm tra và cấu hình trình dịch lần đầu mở

1. Khi phần mềm khởi động lần đầu, cửa sổ nhận diện trình dịch sẽ hiện ra. Em chú ý xem dòng **GNU GCC Compiler** có chữ màu xanh lá cây hoặc hiển thị trạng thái phát hiện thành công hay không, sau đó nhấn **OK**.
2. Một cửa sổ nhỏ hỏi về việc liên kết tệp mã nguồn với phần mềm, em chọn dòng liên kết tất cả rồi nhấn **OK**.

Bước 4: Chạy thử một đoạn mã đơn giản

1. Trên thanh công cụ, em chọn **File** → **New** → **Empty file** (hoặc nhấn tổ hợp phím **Ctrl + Shift + N**).
2. Em lưu tệp lại với đuôi định dạng là .cpp (ví dụ: kiemtra.cpp).
3. Em gõ thử đoạn mã in ra dòng chữ quen thuộc sau:

Dùng cách 1:

```
#include <iostream> //khai báo thư viện nhập và xuất
```

```
using namespace std;

int main() {
  cout << "Xin chao cac ban!";
  return 0;
}
```

4. Em bấm phím **F9** (nút chức năng dịch và chạy). Nếu cửa sổ màu đen xuất hiện dòng chữ *"Xin chao cac ban!"* thì phần mềm đã hoạt động hoàn toàn chính xác.

Dùng cách 2:

```
#include <bits/stdc++.h> //khai báo thư viện chung thay thế tất cả thư viện
using namespace std;

int main() {
  cout << "Xin chao cac ban!";
  return 0;
}
```

2. Cấu trúc chung của chương trình C++:

```
// 1. Khai báo thư viện
#include <iostream>

// 2. Cho phép viết cin, cout thay cho std::cin, std::cout
using namespace std;

// 3. Khai báo hằng, biến toàn cục và hàm phụ nếu cần

// 4. Hàm chính
int main() {
    // Khai báo biến

    // Nhập dữ liệu

    // Xử lí dữ liệu

    // Xuất kết quả

    return 0;
}
```

VD: Nhập hai số nguyên và tính tổng

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b;              // Khai báo biến

    cin >> a >> b;         // Nhập dữ liệu

    int tong = a + b;      // Xử lí dữ liệu

    cout << tong << '\n';  // Xuất kết quả

    return 0;
}
```

Để tiện cho việc khai báo thư viện nhanh, ta thay dòng: #include <bits/stdc++.h>

CẤU TRÚC CHUNG CỦA CHƯƠNG TRÌNH ĐI THI HSG:

```cpp
// 1. Khai báo thư viện tổng hợp
#include <bits/stdc++.h>
using namespace std;

// 2. Khai báo hằng, biến toàn cục nếu cần

// 3. Viết các hàm phụ nếu cần

// 4. Hàm chính
int main() {
    // Tăng tốc độ nhập, xuất
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // Khai báo biến

    // Nhập dữ liệu

    // Xử lí bài toán

    // Xuất kết quả

    return 0;
}
```

BÀI TẬP VẬN DỤNG:

1. Viết chương trình in ra màn hình câu chào các bạn.
2. Nhập vào 2 số nguyên a, b. Tính tổng, hiệu của 2 số vừa nhập.

```cpp
// 1. Khai báo thư viện tổng hợp
#include <bits/stdc++.h>
using namespace std;

// Khai báo hằng, biến toàn cục, hàm phụ nếu cần

int main() {
    // Tăng tốc độ nhập, xuất
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // 1. Khai báo biến
    int a, b;

    // 2. Nhập dữ liệu đúng thứ tự đề bài
    cin >> a >> b;

    // 3. Xử lí bài toán
    int tong = a + b;
    int hieu = a - b;

    // 4. Xuất đúng kết quả đề bài yêu cầu
    cout << tong << '\n';
    cout << hieu ;

    return 0;
}
```

3. BIẾN LÀ GÌ?

a. Định nghĩa:

Biến là một vùng nhớ dùng để lưu dữ liệu trong khi chương trình đang chạy. Mỗi biến có một tên và giá trị của biến có thể thay đổi.

=>Biến giống như một chiếc hộp có tên, dùng để chứa một giá trị và giá trị trong hộp có thể thay đổi.

b. Khai báo:

Kiểu dữ liệu tên_biến;

VD: int a; hoặc int a, b;

c. Gán giá trị cho biến:

tên_biến = giá_trị (biểu thức);

VD: a = 15;

int a = 15; *//vừa khai báo vừa gán giá trị*

a = 15 +5;

4. KIỂU DỮ LIỆU:

Kiểu dữ liệu	Dùng để lưu	Miền giá trị thường gặp	Ví dụ
bool	Đúng / sai	true hoặc false	bool ok = true;
char	Một ký tự	Thường từ -128 đến 127 hoặc 0 đến 255	char c = 'A';
short	Số nguyên nhỏ	-32,768 đến 32,767	short x = 1000;
int	Số nguyên	-2,147,483,648 đến 2,147,483,647	int n = 100;
long long	Số nguyên rất lớn	-9,223,372,036,854,775,808 đến 9,223,372,036,854,775,807	long long n = 10000000000LL;
float	Số thực	Xấp xỉ ±3.4 × 10^38	float x = 3.14f;
double	Số thực chính xác hơn	Xấp xỉ ±1.7 × 10^308	double x = 3.14159;
string	Chuỗi ký tự	Không có miền giá trị cố định như số	string name = "Nhan";

VD:

```
int tuoi = 14;
long long danSo = 10000000000;
double diemTB = 8.75;
char xepLoai = 'A';
string hoTen = "Nguyen Van An";
bool dat = true;
```

5. CÁC PHÉP TOÁN TRONG C++

a. Phép toán số học:

Phép toán	Ký hiệu	Ý nghĩa	Ví dụ
Cộng	+	Cộng hai giá trị	a + b
Trừ	-	Lấy giá trị thứ nhất trừ giá trị thứ hai	a - b
Nhân	*	Nhân hai giá trị	a * b
Chia	/	Chia hai giá trị	a / b

Chia lấy dư	%	Lấy phần dư của phép chia số nguyên	a % b

Lưu ý: * Nếu a và b đều là số nguyên thì a / b là phép chia nguyên.

* Nếu ít nhất một trong hai toán hạng là số thực (float hoặc double) thì phép / sẽ cho kết quả là số thực.

b. Phép toán gán:

Phép toán	Ký hiệu	Ý nghĩa	Tương đương
Gán	=	Gán giá trị	a = 5
Cộng rồi gán	+=	Cộng thêm vào biến	a += b ⇔ a = a + b
Trừ rồi gán	-=	Trừ rồi lưu lại	a -= b ⇔ a = a - b
Nhân rồi gán	*=	Nhân rồi lưu lại	a *= b ⇔ a = a * b
Chia rồi gán	/=	Chia rồi lưu lại	a /= b ⇔ a = a / b
Chia dư rồi gán	%=	Lấy dư rồi lưu lại	a %= b ⇔ a = a % b

BÀI TẬP VẬN DỤNG:

Bài 1: Nhập vào 2 biến a, b. Tính tổng , hiệu, tích, thương.

Bài 2: Nhập 3 biến x, y, z. Tính trung bình cộng của 3 số vừa nhập.

Bài 3: Nhập điểm Toán, Văn, Anh của em. Tính điểm trung bình của em (Văn, Toán hệ số 2).

Bài 4: Nhập chiều dài và chiều rộng hình chữ nhật ABCD. Tính Chu vi và Diện.

Bài 5: Tìm phần dư của phép chia a /b (a, b là 2 số nguyên khác 0)

6. CÂU LỆNH ĐIỀU KIỆN:

a. Dạng thiếu:

```
if (điều_kiện) {
    câu_lệnh;
}
```

Ý nghĩa: nếu điều_kiện đúng thì thực hiện câu_lệnh. Nếu sai thì bỏ qua.

VD:

```
int a = 10;

if (a > 0) {
    cout << "a la so duong";
}
```

b. Dạng đủ: if ... else

```
if (điều_kiện) {
    câu_lệnh_1;
}
else {
    câu_lệnh_2;
}
```

Ý nghĩa:

- Nếu điều kiện đúng → thực hiện câu_lệnh_1.
- Nếu điều kiện sai → thực hiện câu_lệnh_2.

VD:

```
int a;
cin >> a;

if (a % 2 == 0) {
    cout << "a la so chan";
}
else {
    cout << "a la so le";
}
```

c. Phép toán so sánh:

Kết quả của phép so sánh là true hoặc false

Phép toán	Ký hiệu	Ý nghĩa	Ví dụ
Bằng	==	Hai giá trị bằng nhau	a == b
Khác	!=	Hai giá trị khác nhau	a != b
Lớn hơn	>	Giá trị bên trái lớn hơn bên phải	a > b
Nhỏ hơn	<	Giá trị bên trái nhỏ hơn bên phải	a < b
Lớn hơn hoặc bằng	>=	Lớn hơn hoặc bằng	a >= b
Nhỏ hơn hoặc bằng	<=	Nhỏ hơn hoặc bằng	a <= b

VD:

```
int a = 10, b = 5;

cout << (a > b);   // 1
cout << (a == b);  // 0
```

d. Phép toán logic:

Phép toán logic dùng để kết hợp nhiều điều kiện.

Phép toán	Ký hiệu	Ý nghĩa	Ví dụ
Và	&&	Đúng khi cả hai điều kiện đều đúng	a > 0 && a < 10
Hoặc	||	Đúng khi chỉ cần một điều kiện đúng	a > 0 || a < 10
Phủ định	!	Đảo ngược kết quả đúng/sai	!(a > 0)

VD:

```
int a = 7;

if (a >= 1 && a <= 10){
cout << "a nam trong doan 1 den 10";
}
```

BÀI TẬP VẬN DỤNG:

Bài 1: Nhập vào 1 số nguyên N (n khác 0). Hỏi N là số âm hay số dương.

Bài 2: Nhập vào số nguyên X. Hỏi X là số chẵn hay số lẻ.

Bài 3: Cho 2 số a, b. Tìm số lớn nhất. Nếu không có hãy in ra “a = b”

Bài 4: Nhập vào tuổi của học sinh. Nếu tuổi từ 15 trở lên thì in: Du tuoi, ngược lại in: Chua du tuoi.

VD:

```
Input: 14
Output: Chua du tuoi
```

Bài 5: Nhập vào 2 số bất kỳ. Hỏi tổng của chúng có chia hết cho 3 và 5 không?

VD: Input: 3 2

Output: Khong chia het

7. ĐỌC GHI FILE TRONG THI HSG:

a. Lệnh đọc ghi file:

- Ghi file: freopen("TENBAI.INP", "r", stdin);

- Đọc file: freopen("TENBAI.OUT", "w", stdout);

Lưu ý:

- r là đọc (read), w là ghi (write), TENBAI do đề bài yêu cầu
- 2 file này lưu chung vào cùng 1 thư mục với file bài làm.
- Chỉ cần tạo file TENBAI.INP ban đầu, file TENBAI.OUT chương trình tự tạo

b. Cấu trúc chung HS cần nhớ:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    //Tăng tốc độ nhập xuất dữ liệu
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    freopen("TENBAI.INP", "r", stdin);
    freopen("TENBAI.OUT", "w", stdout);

    // Nhập dữ liệu
    // cin >> ...

    // Xử lý

    // Xuất kết quả
    // cout << ...

    return 0;
}
```

Qui trình chuẩn: **Đọc tên file → xác định Input → xác định Output → chọn kiểu dữ liệu → viết xử lý → xuất đúng định dạng.**

8. CÂU LỆNH LẶP

8.1 Vòng lặp for (Biết trước số lần lặp)

Dùng khi đã xác định rõ số lần cần thực hiện khối lệnh (duyệt dãy số, duyệt chỉ mục mảng).

Cú pháp:

```
for (khởi_tạo; điều_kiện; bước_nhảy) {
// Khối lệnh thực thi
}
```

- **Luồng chạy:**
    1. Thực hiện khởi_tạo (chỉ 1 lần duy nhất).
    2. Kiểm tra điều_kiện: Nếu true → thực thi khối lệnh; nếu false → thoát vòng lặp.
    3. Sau khi khối lệnh chạy xong → thực hiện bước_nhảy.
    4. Quay lại bước 2.

Ví dụ: Tính tổng các số từ 1 đến N: (Tong = 1+ 2 + 3 +...+ N)

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = 10;
    int tong = 0;
    for (int i = 1; i <= n; ++i) {
        tong += i;
    }

    cout << "Tong: " << tong << "\n"; // 55

    return 0;
}
```

8.2. **<u>Vòng lặp while (Chưa biết trước số lần lặp)</u>**

Dùng khi số lần lặp phụ thuộc vào một điều kiện động, **kiểm tra điều kiện trước khi thực hiện**.

Cú pháp:

```cpp
while (điều_kiện) {
// Khối lệnh thực thi
}
```

Ví dụ: Tìm chữ số đầu tiên của một số nguyên dương:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);


    int n = 8352;
    while (n >= 10) {
        n /= 10; // Cắt bỏ chữ số hàng đơn vị sau mỗi bước
    }
    cout << "Chu so dau tien: " << n << "\n"; // In ra 8


    return 0;
}
```

8.3. Vòng lặp do-while (Luôn thực hiện ít nhất 1 lần)

Tương tự while, nhưng **kiểm tra điều kiện ở cuối vòng lặp**.

Cú pháp:

```cpp
do {
    // Khối lệnh thực thi
} while (điều_kiện); // Lưu ý: bắt buộc có dấu chấm phẩy ';' ở cuối
```

- **Đặc điểm:** Khối lệnh luôn được thực thi **ít nhất một lần**, bất kể điều kiện ban đầu đúng hay sai.
- **Ứng dụng phổ biến:** Kiểm tra hợp lệ dữ liệu nhập từ bàn phím (Input Validation) hoặc hiển thị Menu lựa chọn.

Ví dụ: Bắt buộc người dùng nhập số nguyên dương:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);


    int n;
    do {
        cout << "Nhap so nguyen duong (n > 0): ";
        cin >> n;
    } while (n <= 0); // Nhập sai thì tiếp tục lặp lại

    cout << "Ban da nhap dung: " << n << "\n";


    return 0;
}
```

8.4. Vòng lặp Range-based for (Duyệt theo phần tử - từ C++11)

Dùng để duyệt qua toàn bộ các phần tử của một mảng tĩnh, chuỗi (std::string), hoặc container (std::vector, std::set).

Cú pháp:

```cpp
for (kiểu_dữ_liệu bien : tap_hop) {
    // Khối lệnh
}
```

Ví dụ:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);



    vector<int> a = {2, 4, 6, 8, 10};

    // Dung 'auto&' neu muon tham chieu/sua gia tri, 'const auto&' neu chi doc
    for (int x : a) {
        cout << x << " ";
    }



    return 0;
}
```

8.5. Hai lệnh điều khiển quan trọng trong vòng lặp

- **break:** Ngay lập tức kết thúc vòng lặp gần nó nhất và thoát ra ngoài.

```cpp
for (int i = 1; i <= 10; ++i) {
if (i == 5) break; // Gap 5 la dung toan bo vong lap
cout << i << " ";  // In: 1 2 3 4
}
```

- **continue:** Bỏ qua các lệnh còn lại của **lần lặp hiện tại** và nhảy sang lần lặp kế tiếp.

```cpp
for (int i = 1; i <= 5; ++i) {
if (i == 3) continue; // Bo qua so 3
cout << i << " ";     // In: 1 2 4 5
}
```

Bảng tóm tắt so sánh

Cấu trúc	Vị trí kiểm tra điều kiện	Số lần lặp tối thiểu	Tình huống sử dụng tối ưu
for	Đầu vòng lặp	0 lần	Đã biết trước số lần lặp, duyệt mảng bằng chỉ mục.
while	Đầu vòng lặp	0 lần	Chưa rõ số lần lặp, lặp theo trạng thái logic (nhập file, xử lý số).

do-while	Cuối vòng lặp	1 lần	Bắt buộc chạy ít nhất 1 lần (nhập dữ liệu có điều kiện, menu).
Range-based	Tự động	0 lần	Duyệt trọn vẹn từng phần tử của mảng, vector, string.

BÀI TẬP VẬN DỤNG

Mức độ 1: Nhận biết & Thông hiểu (Cơ bản)

Bài 1: Tính tổng dãy số liên tiếp

- **Đề bài:** Nhập vào một số nguyên dương n ($1 \leq n \leq 10^6$). Tính tổng $S = 1 + 2 + 3 + \cdots + n$.
- **Mục tiêu rèn luyện:** Cấu trúc vòng lặp for cơ bản, biến tích lũy, chú ý kiểu dữ liệu (tránh tràn số int).
- **Ví dụ:**

Input	Output
5	15

CODE MẪU

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);


    int n;
    cin >> n;

    // Dùng kiểu long long cho biến tong vì n lên đến 10^6:
    // Tổng lớn nhất: 10^6 * (10^6 + 1) / 2 ≈ 5 * 10^11 (vượt giới hạn int 2*10^9)
    long long tong = 0;

    for (int i = 1; i <= n; ++i) {
        tong += i;
    }

    cout << tong << "\n";


    return 0;
}
```

KẾT QUẢ SAU KHI CHẠY CHƯƠNG TRÌNH:

```
6500000
21125003250000
```

Bài 2: Bảng cửu chương rút gọn

- **Đề bài:** Nhập vào số nguyên dương k ($1 \leq k \leq 9$). In ra bảng nhân của số k từ 1 đến 10, mỗi dòng một phép tính theo định dạng k x i = ket_qua.

- **Mục tiêu rèn luyện:** Vòng lặp for với số bước cố định, định dạng xuất dữ liệu.
- **Ví dụ:**

Input	Output
3	3 x 1 = 3 3 x 2 = 6 ... 3 x 10 = 30

Bài 3: Đếm số lượng và tính tổng các chữ số

- **Đề bài:** Nhập vào một số nguyên dương n ($1 \leq n \leq 10^9$). Hãy đếm xem n có bao nhiêu chữ số và tính tổng các chữ số của n.
- **Mục tiêu rèn luyện:** Vòng lặp while, phép chia lấy dư % 10 và chia nguyên / 10 để tách số.
- **Ví dụ:**
    - **Input:** 2024
    - **Output:** So chu so: 4, Tong chu so: 8

Mức độ 2: Vận dụng cơ bản (Trung bình)

Bài 4: Kiểm tra tính hợp lệ khi nhập (Input Validation)

- **Đề bài:** Viết chương trình yêu cầu người dùng nhập điểm thi của một học sinh (thang điểm từ 0 đến 10). Nếu nhập sai (nhỏ hơn 0 hoặc lớn hơn 10), chương trình bắt buộc người dùng nhập lại cho đến khi đúng thì in ra: Diem hop le: [diem].
- **Mục tiêu rèn luyện:** Ứng dụng vòng lặp do-while trong kiểm soát dữ liệu đầu vào.
- **Ví dụ:**
    - **Input:** -2 → 15 → 8
    - **Output:** Diem hop le: 8

Bài 5: Đảo ngược một số nguyên

- **Đề bài:** Nhập vào số nguyên dương n ($1 \leq n \leq 10^9$). Hãy tạo và in ra số đảo ngược của n (không in các số 0 vô nghĩa ở đầu).
- **Mục tiêu rèn luyện:** Vòng lặp while, thuật toán ghép số ngược: res = res * 10 + n % 10.
- **Ví dụ:**
    - **Input:** 12345
    - **Output:** 54321

Bài 6: Tìm số lớn nhất và nhỏ nhất trong luồng dữ liệu

- **Đề bài:** Dòng đầu tiên nhập số lượng phần tử n ($1 \leq n \leq 10^5$). Dòng tiếp theo lần lượt nhập n số nguyên. Hãy tìm giá trị lớn nhất (Max) và nhỏ nhất (Min) trong n số đó mà **không lưu trữ vào mảng**.

- **Mục tiêu rèn luyện:** Xử lý dữ liệu động trực tiếp (Streaming/Online algorithm): nhập đến đâu cập nhật Max/Min đến đó.
- **Ví dụ:**

Input	Output
5 7 2 9 1 5	Max = 9, Min = 1

Bài 7: Kiểm tra số nguyên tố

- **Đề bài:** Nhập vào số nguyên n ($n \leq 10^9$). Kiểm tra xem n có phải là số nguyên tố hay không. Nếu có in YES, ngược lại in NO.
- **Mục tiêu rèn luyện:** Tối ưu hóa vòng lặp (chỉ lặp từ 2 đến $\sqrt{n}$), lệnh break ngắt vòng lặp sớm.
- **Ví dụ:**
  - **Input:** 17 → **Output:** YES
  - **Input:** 21 → **Output:** NO

Mức độ 3: Vận dụng cao (Nâng cao)

Bài 8: Ước chung lớn nhất (UCLN) và Bội chung nhỏ nhất (BCNN)

- **Đề bài:** Nhập vào 2 số nguyên dương a và b ($1 \leq a, b \leq 10^9$). Tìm UCLN và BCNN của hai số.
- **Mục tiêu rèn luyện:** Cài đặt thuật toán Euclid bằng vòng lặp while không đệ quy: while (b != 0) { int r = a % b; a = b; b = r; }.
- **Ví dụ:**
  - **Input:** 12 18
  - **Output:** UCLN = 6, BCNN = 36

Bài 9: Vẽ hình tam giác số (Vòng lặp lồng nhau)

- **Đề bài:** Nhập vào chiều cao h ($1 \leq h \leq 9$). In ra tam giác vuông cân dạng số theo quy luật:
  - Dòng 1: 1
  - Dòng 2: 1 2
  - Dòng 3: 1 2 3
  - ...
  - Dòng h: 1 2 3 ... h
- **Mục tiêu rèn luyện:** Tư duy 2 vòng lặp for lồng nhau (Nested loops): vòng ngoài quản lý số dòng, vòng trong quản lý số cột/giá trị in ra.
- **Ví dụ:**

Input	Output
4	1 1 2

	1 2 3 1 2 3 4

Bài 10: Phân tích ra thừa số nguyên tố

- **Đề bài:** Nhập vào một số nguyên dương n ($2 \leq n \leq 10^9$). Phân tích n thành tích các thừa số nguyên tố.
- **Mục tiêu rèn luyện:** Kết hợp vòng lặp for và while, tư duy giảm dần bài toán mà không cần lưu danh sách các thừa số.
- **Ví dụ:**
  - **Input:** 60
  - **Output:** 2 * 2 * 3 * 5 (hoặc 2^2 * 3^1 * 5^1)

MẢNG (ARRAY)

1. Mở đầu: Vì sao chúng ta cần Mảng?

- **Bài toán thực tế:** Giả sử lớp mình có 40 bạn, cần nhập điểm môn Tin học và tính điểm trung bình cả lớp.
- **Cách cũ (khi chưa học mảng):** Ta phải khai báo 40 biến đơn:

```
double d1, d2, d3, ..., d40;
cin >> d1 >> d2 >> d3 >> ... >> d40;
```

- → Viết code rất dài dòng, dễ nhầm và **không thể dùng vòng lặp để xử lý tự động** được.
- **Giải pháp:** C++ cung cấp **Mảng (Array)** — một tập hợp nhiều biến có **cùng kiểu dữ liệu**, được xếp **liền kề nhau** trong bộ nhớ máy tính.

Hình ảnh liên tưởng: *Hãy tưởng tượng mảng giống như một **dãy tủ gửi đồ ở siêu thị** gồm nhiều ngăn liên tiếp nhau. Cả dãy tủ có chung một cái tên (ví dụ: tủ a), mỗi ngăn chứa một đồ vật (giá trị) và được đánh số thứ tự từ 0 để nhận biết.*

2. Cú pháp khai báo và cách đánh số thứ tự (Index)

Cú pháp:

kieu_du_lieu ten_mang[**kich_thuoc_toi_da**];

Ví dụ:

```
int a[100];       // Khai báo một dãy tối đa 100 số nguyên
double diem[45]; // Khai báo một dãy tối đa 45 điểm số thực
```

Quy tắc "vàng" về số thứ tự (Chỉ mục - Index):

Trong C++, các ngăn trong mảng luôn **bắt đầu từ số 0**, không phải từ 1:

- Nếu khai báo int a[5]; mảng gồm đúng 5 ô: a[0], a[1], a[2], a[3], a[4].
- **Cảnh báo lỗi nghiêm trọng:** Truy cập a[5] là truy cập ra ngoài vùng nhớ được cấp phát (lỗi **Out of Bounds**), chương trình sẽ chạy sai hoặc bị dừng đột ngột (Crash).

3. Bộ ba thao tác cơ bản nhất: Nhập — Xuất — Tính toán

Vì chỉ số mảng chạy liên tục từ **0 đến n - 1**, ta luôn kết hợp mảng với **vòng lặp for**.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);


    int n;
    cout << "Nhap so luong phan tu: ";
    cin >> n;

    int a[100]; // Tao mang chua toi da 100 so nguyen

    // 1. NHẬP DỮ LIỆU VÀO MẢNG
    for (int i = 0; i < n; ++i) {
        cin >> a[i]; // Nhap gia tri cho phan tu thu i
    }

    // 2. IN MẢNG RA MÀN HÌNH
    cout << "Cac phan tu vua nhap la: ";
    for (int i = 0; i < n; ++i) {
        cout << a[i] << " ";
    }
    cout << "\n";

    // 3. TÍNH TỔNG CÁC PHẦN TỬ TRONG MẢNG
    long long tong = 0;
    for (int i = 0; i < n; ++i) {
        tong += a[i];
    }
    cout << "Tong cac phan tu: " << tong << "\n";


    return 0;
}
```

4. Hai bài toán "kinh điển" khi mới học mảng

Bài toán 1: Tìm phần tử lớn nhất có trong mảng (Max)

- **Ý tưởng:** Coi phần tử đầu tiên a[0] là nhà vô địch tạm thời (max_val). Sau đó đi so sánh lần lượt với các phần tử từ a[1] đến a[n - 1]. Ai lớn hơn thì cướp ngôi vô địch.

```cpp
int n;
cin >> n;
cout << "So luong phan tu: " << n << "\n";


int a[100]; // Tao mang chua toi da 100 so nguyen

// 1. NHẬP DỮ LIỆU VÀO MẢNG
for (int i = 0; i < n; ++i) {
    cin >> a[i]; // Nhap gia tri cho phan tu thu i
}
cout << "\n";

// 2. IN MẢNG RA MÀN HÌNH
cout << "Cac phan tu vua nhap la: ";
for (int i = 0; i < n; ++i) {
    cout << a[i] << " ";
}
cout << "\n";

// 3. Tìm phần tử lớn nhất (Max)
int max_val = a[0];
for (int i = 1; i < n; ++i) {
    if (a[i] > max_val) {
        max_val = a[i]; // Cap nhat ky luc moi
    }
}
cout << "Gia tri lon nhat la: " << max_val << "\n";
```

Kết quả khi chạy chương trình:

```
5
6
12
3
9
25
So luong phan tu: 5

Cac phan tu vua nhap la: 6 12 3 9 25
Gia tri lon nhat la: 25
```

Bài toán 2: Đếm số lượng số chẵn trong mảng

- **Ý tưởng:** Dùng một biến đếm khởi tạo bằng 0. Đi qua từng ô, kiểm tra điều kiện chia hết cho 2 (a[i] % 2 == 0).

```cpp
int n;
cin >> n;
cout << "So luong phan tu: " << n << "\n";


int a[100]; // Tao mang chua toi da 100 so nguyen

// 1. NHẬP DỮ LIỆU VÀO MẢNG
for (int i = 0; i < n; ++i) {
    cin >> a[i]; // Nhap gia tri cho phan tu thu i
}
cout << "\n";

// 2. IN MẢNG RA MÀN HÌNH
cout << "Cac phan tu vua nhap la: ";
for (int i = 0; i < n; ++i) {
    cout << a[i] << " ";
}
cout << "\n";

// 3. Đếm số lượng số chẵn trong mảng
int dem_chan = 0;
for (int i = 0; i < n; ++i) {
    if (a[i] % 2 == 0) {
        dem_chan++;
    }
}
cout << "So luong so chan la: " << dem_chan << "\n";
```

Kết quả sau khi chạy chương trình:

```
5
6
3
9
1
12
So luong phan tu: 5

Cac phan tu vua nhap la: 6 3 9 1 12
So luong so chan la: 2
```

5. Ba lưu ý cốt lõi nhắc học sinh cần nhớ

a) **Chỉ số chạy từ** 0 **đến** $n-1$**:** Muốn duyệt hết n phần tử, điều kiện lặp luôn là for (int i = 0; i < n; ++i) (dấu < n, tuyệt đối không dùng <= n).

b) **Kích thước mảng tĩnh:** Số lượng phần tử khai báo trong ngoặc vuông [ ] phải là một **hằng số đủ lớn** (ví dụ đề bài bảo $N \leq 10^5$ thì khai báo int a[100005];).

c) **Mảng không tự xóa rác:** Nếu khai báo cục bộ bên trong hàm main, các ô nhớ ban đầu chứa các giá trị rác ngẫu nhiên. Vì vậy phải nhập hoặc gán giá trị trước khi đem đi tính toán.

BÀI TẬP VẬN DỤNG

Bài 1: Nhập và in mảng theo thứ tự đảo ngược

- **Mục tiêu:** Giúp học sinh thành thạo kỹ năng duyệt chỉ số mảng theo chiều ngược lại (từ $n-1$ về 0).
- **Đề bài:** Nhập vào số nguyên dương n ($1 \leq n \leq 1000$) và dãy gồm n số nguyên. Hãy in ra dãy số đó theo thứ tự từ phần tử cuối cùng về phần tử đầu tiên.
- **Ví dụ:**

Input	Output
5 3 8 1 9 4	4 9 1 8 3

CODE MẪU:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    int a[1005]; // Khai bao mang du rong de chua toi da 1000 phan tu

    // Nhap mang tu dau den cuoi
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    // Duyet nguoc tu chi so cuoi cung (n - 1) ve chi so dau tien (0)
    for (int i = n - 1; i >= 0; --i) {
        cout << a[i] << " ";
    }
    cout << "\n";


    return 0;
}
```

Bài 2: Đếm và tính trung bình cộng các số dương

- **Mục tiêu:** Kết hợp duyệt mảng với câu lệnh điều kiện if, rèn luyện cách ép kiểu số thực khi tính trung bình.
- **Đề bài:** Cho dãy gồm n số nguyên ($1 \leq n \leq 1000$, các phần tử nằm trong khoảng $[-10^4, 10^4]$). Hãy:
    1. Đếm xem có bao nhiêu số mang giá trị dương (> 0).
    2. Tính trung bình cộng của các số dương đó (nếu không có số dương nào thì in ra 0).
- **Ví dụ:**

Input	**Output**
5 -3 4 0 6 -2	So luong so duong: 2 Trung binh cong: 5

Bài 3: Tìm vị trí của phần tử nhỏ nhất (Min)

- **Mục tiêu:** Phân biệt rõ giữa **giá trị** của phần tử (a[i]) và **vị trí/chỉ mục** của phần tử (i).
- **Đề bài:** Cho dãy n số nguyên ($1 \leq n \leq 1000$). Hãy tìm giá trị nhỏ nhất trong mảng và chỉ ra vị trí xuất hiện đầu tiên của nó (theo thứ tự vị trí từ 1 đến n cho người dùng phổ thông dễ nhìn).
- **Ví dụ:**

Input	**Output**
6 15 7 24 3 8 3	Gia tri nho nhat: 3 tai vi tri: 4

Bài 4: Tìm kiếm phần tử X (Tìm kiếm tuyến tính)

- **Mục tiêu:** Rèn luyện thuật toán tìm kiếm cơ bản trên mảng và kỹ thuật dùng cờ hiệu (flag).
- **Đề bài:** Cho mảng gồm n số nguyên và một số nguyên X. Kiểm tra xem X có xuất hiện trong mảng hay không. Nếu có, in ra YES và vị trí xuất hiện đầu tiên của X. Nếu không, in ra NO.
- **Ví dụ:**

Input 1	5 7 2 5 7 9 7	**Input 2**	4 10 1 3 5 7
Output 1	YES 3	**Output 2**	NO

Bài 5: Kiểm tra mảng đối xứng (Palindromic Array)

- **Mục tiêu:** Rèn luyện tư duy so sánh đối xứng giữa hai đầu mảng (a[i] với a[n - 1 - i]).

- **Đề bài:** Một mảng được gọi là đối xứng nếu đọc từ trái sang phải hay từ phải sang trái đều thu được thứ tự các phần tử như nhau. Cho mảng n số nguyên, kiểm tra xem mảng đó có đối xứng hay không (In YES nếu đúng, ngược lại in NO).
- **Ví dụ:**

Input 1	5 1 2 3 2 1	**Input 2**	4 1 2 2 3
Output 1	YES	**Output 2**	NO

Mảng đếm phân phối
(Frequency Array)

1. Đặt vấn đề: Bài toán kiểm phiếu bầu cử

- **Tình huống**: Lớp bầu lớp trưởng, có **3 ứng viên mang mã số: $\mathbf{1}, \mathbf{2}, \mathbf{3}$**. Có 40 phiếu bầu (mỗi phiếu ghi số 1,2 hoặc 3). Làm sao để kiểm phiếu nhanh nhất?
- **Cách ngây thơ (duyệt tìm kiếm):**
  - Cầm từng lá phiếu, rồi lật lại từ đầu để đếm xem số 1 xuất hiện bao nhiêu lần, số 2 bao nhiêu lần... → Rất chậm, mất nhiều thời gian lặp đi lặp lại.
- **Cách thực tế ngoài đời:**
  - Chuẩn bị 3 chiếc hộp dán nhãn: Hộp 1, Hộp 2, Hộp 3.
  - Bốc lá phiếu ghi số X → Thả ngay vào hộp mang nhãn số X.
  - Cuối cùng, chỉ cần đếm số phiếu trong từng hộp là xong!

<u>Giải quyết tình huống:</u>

a. Cách ngây thơ (duyệt tìm kiếm)

```
int n;
cin >> n;

int dem1 = 0;
for (int i = 0; i < n; i++){
    if (a[i] == 1){
        dem1++;
    }
}

int dem2 = 0;
for (int i = 0; i < n; i++){
    if (a[i] == 2){
        dem2++;
    }
}

int dem3 = 0;
for (int i = 0; i < n; i++){
    if (a[i] == 3){
        dem3++;
    }
}
```

Với cách này, mỗi lần đếm 1 ứng viên, chương trình sẽ lặp n lần => rất lâu.

b. Cách thực tế ngoài đời (Frequency Array)

Tư tưởng cốt lõi của Frequency Array: Biến giá trị của phần tử thành chỉ số (index) của mảng đếm.

- Giá trị cần đếm: x
- Ngăn đếm tương ứng: **cnt[x]**
- Mỗi lần gặp số x: tăng ngăn đó lên 1 đơn vị: **cnt[x]++**

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    int cnt[4] = {0};

    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;

        cnt[x]++;
    }

    for (int i = 1; i <= 3; i++) {
        cout << "Ung vien " << i << ": " << cnt[i] << " phieu\n";
    }

    return 0;
}
```

Kết quả sau khi chạy chương trình:

```
5
2 3 1 3 2
Ung vien 1: 1 phieu
Ung vien 2: 2 phieu
Ung vien 3: 2 phieu
```

Minh họa bằng bảng:

Chỉ số	cnt[0]	cnt[1]	cnt[2]	cnt[3]
Ý nghĩa	Không dùng	Ứng viên 1	Ứng viên 2	Ứng viên 3
Số phiếu	0	1	2	2

2. So sánh độ phức tạp: Vì sao kỹ thuật này cực mạnh?

Giả sử cần đếm số lần xuất hiện của các phần tử trong dãy có N số (giá trị tối đa là M):

Cách làm	Cơ chế	Độ phức tạp thời gian	Khả năng chạy với N=105
Cách ngây thơ (duyệt tìm kiếm)	Với mỗi số, quét lại cả mảng để đếm	$\mathcal{O}(N^2)$	Mất ~10 giây → Bị quá thời gian (TLE)
Frequency Array	Đọc đến đâu, tăng cnt[x]++ trực tiếp	$\mathcal{O}(N+M)$	Mất ~0.005 giây → Chạy tức thì

3. Ba cạm bẫy học sinh bắt buộc phải ghi nhớ

a) **Giới hạn giá trị của phần tử ($A[i]$):**

- Mảng trong C++ thường chỉ khai báo được kích thước tối đa khoảng 10^6 đến 10^7 phần tử (tương đương 40MB RAM).
- Do đó, kỹ thuật này chỉ dùng được khi giá trị các phần tử $\leq 10^6$. Nếu đề cho giá trị $A[i] \leq 10^9$ thì không thể dùng mảng đếm trực tiếp mà phải dùng kỹ thuật nén số hoặc std::map.

b) **Khai báo mảng ở ngoài hàm main:**

- Lưu ý khi khai báo mảng lớn: Mảng cục bộ được khai báo bên trong main thường sử dụng bộ nhớ Stack. Nếu mảng quá lớn, chương trình có thể vượt giới hạn Stack và xảy ra Stack Overflow. Trong các bài thi HSG, nên khai báo các mảng lớn ở phạm vi toàn cục để an toàn hơn.

  VD:

```
#include <bits/stdc++.h>
using namespace std;

const int MAX = 1000000;
int cnt[MAX + 5];

int main() {
    ...
}
```

c) **Trường hợp có số âm:**

- **Chỉ số mảng C++ không thể là số âm (ví dụ cnt[-5] là sai).**

- **Cách xử lý:** Cộng thêm một lượng độ dời (Offset). Ví dụ nếu giá trị nằm trong đoạn $[-10^5, 10^5]$, ta cộng thêm 10^5 vào giá trị: lưu số x tại cnt[x + 100000].

BÀI TẬP VẬN DỤNG:

Bài 1: Đếm số lượng phần tử phân biệt (Khác nhau đôi một)

- **Mục tiêu:** Giúp học sinh làm quen với việc dùng mảng đánh dấu để kiểm tra xem một giá trị đã từng xuất hiện hay chưa.
- **Đề bài:** Cho dãy gồm N số nguyên dương ($1 \leq N \leq 10^5$, các phần tử A_i thỏa mãn $1 \leq A_i \leq 10^5$). Hãy đếm xem trong dãy có bao nhiêu số khác nhau đôi một.
- **Ví dụ:**

Input	**Output**
6 **3 5 3 2 5 8**	**4**

Phân tích & Hướng dẫn giải

- **Ý tưởng:**
  1. Khai báo mảng đếm cnt toàn cục (được tự động gán toàn bộ bằng 0).
  2. Đọc từng số x, tăng cnt[x]++.
  3. Duyệt chỉ số v từ giá trị nhỏ nhất đến giá trị lớn nhất: nếu cnt[v] > 0 nghĩa là giá trị v có mặt trong dãy → tăng biến đếm số phân biệt lên 1.
- **Độ phức tạp:** Thời gian $\mathcal{O}(N + \max(A_i))$, Bộ nhớ $\mathcal{O}(\max(A_i))$.

Code mẫu C++

```cpp
using namespace std;

// Khai bao mang dem ngoai ham main de tranh tran Stack va tu dong gan = 0
const int MAX_VAL = 100005;
int cnt[MAX_VAL];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    int max_val = 0;
    for (int i = 0; i < n; ++i) {
        int x;
        cin >> x;
        cnt[x]++;
        if (x > max_val) max_val = x;
    }

    int so_phan_biet = 0;
    for (int v = 1; v <= max_val; ++v) {
        if (cnt[v] > 0) {
            so_phan_biet++;
        }
    }

    cout << so_phan_biet << "\n";
    return 0;
}
```

Bài 2: Phần tử áp đảo (Tìm giá trị xuất hiện nhiều nhất)

- **Mục tiêu:** Rèn luyện thao tác vừa đếm vừa cập nhật giá trị cực trị (Max Frequency), xử lý trường hợp bằng nhau ưu tiên giá trị nhỏ hơn.
- **Đề bài:** Cho dãy số gồm N phần tử ($1 \leq N \leq 10^5$, $1 \leq A_i \leq 10^5$). Hãy tìm giá trị xuất hiện nhiều lần nhất trong dãy. Nếu có nhiều giá trị có cùng số lần xuất hiện nhiều nhất, hãy in ra giá trị nhỏ nhất trong số đó.
- **Ví dụ:**

Input	Output
7 **4 2 7 2 4 9 2**	**2**

Phân tích & Hướng dẫn giải

- Ý tưởng:
    1. Nhập dãy và đếm tần số qua mảng cnt[x]++.
    2. Duyệt qua tất cả các giá trị v từ nhỏ đến lớn ($1 \rightarrow \max$).

3. Duy trì 2 biến: max_freq (số lần xuất hiện kỷ lục) và ans (giá trị mang kỷ lục đó).
4. Vì duyệt từ giá trị nhỏ đến lớn, ta dùng điều kiện ngặt cnt[v] > max_freq để tự động chọn giá trị nhỏ hơn khi có sự hòa điểm tần suất.

Bài 3: Một giáo viên phát cho học sinh các thẻ được đánh số từ 1 đến n. Theo đúng quy định, mỗi số từ 1 đến n (1 ≤ n ≤ 100000) phải xuất hiện đúng hai lần. Tuy nhiên trong quá trình phát thẻ, có đúng một số chỉ xuất hiện một lần. Hãy tìm số đó.

Gợi ý: Dữ liệu tổng: 2 × n – 1

Input: 5 1 3 2 5 1 2 3 5 4	**Output: 4**

SẮP XẾP MẢNG

1. Sắp xếp là gì và vì sao phải sắp xếp?

- **Định nghĩa:** Sắp xếp là việc sắp đặt lại thứ tự các phần tử trong mảng theo một trật tự nhất định — thông thường là tăng dần (không giảm) hoặc giảm dần (không tăng).
- **Ứng dụng:**
  - Tra cứu từ điển nhanh hơn (tìm kiếm nhị phân $\mathcal{O}(\log N)$ chỉ chạy được trên mảng đã sắp xếp).
  - Tìm giá trị lớn thứ nhì, nhỏ thứ ba, hoặc tìm trung vị dãy số.
  - Gom các phần tử giống nhau về cạnh nhau để đếm hoặc loại bỏ phần tử trùng.

VD:

- **TA CÓ MẢNG A (5 phần tử hoặc n =5)**

a[0]	a[1]	a[2]	a[3]	a[4]
5	2	8	1	3

- **SAU KHI SẮP XẾP TĂNG DẦN:**

a[0]	a[1]	a[2]	a[3]	a[4]
1	2	3	5	8

- **SAU KHI SẮP XẾP GIẢM DẦN:**

a[0]	a[1]	a[2]	a[3]	a[4]
8	5	3	2	1

2. LỆNH SẮP XẾP TRONG C++:

a. Sắp xếp tăng dần:

sort(a, a + n);

Giải thích:

- a => a[0]
- n : số phần tử mảng (trong VD trên n= 5)

Ý nghĩa: Sắp xếp từ a[0] đến trước a[5] tức là **Sắp xếp từ a[0] đến a[4]**

CODE MINH HỌA:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    int a[100005];

    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    sort(a, a + n);

    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }

    return 0;
}
```

KẾT QUẢ SAU KHI CHẠY:

```
5
5 2 8 1 3
1 2 3 5 8
Process returned 0 (0x0)   execution time : 34.834 s
Press any key to continue.
```

b. Sắp xếp giảm dần:

sort(a, a + n, greater<kieu_du_lieu>());

Giải thích:

- a => a[0]
- n : số phần tử mảng (trong VD trên n= 5)
- <kieu_du_lieu> : kiểu dữ liệu của mảng
- greater : làm cho phần tử lớn được xếp trước phần tử nhỏ.

Ý nghĩa: Sắp xếp giảm dần các số trong mảng

CODE MINH HỌA:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    int a[100005];

    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    sort(a, a + n, greater<int>());

    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }

    return 0;
}
```

KẾT QUẢ SAU KHI CHẠY:

```
5
5 2 8 1 3
8 5 3 2 1
Process returned 0 (0x0)   execution time : 26.259 s
Press any key to continue.
```

BÀI TẬP VẬN DỤNG

Bài 1: Sắp xếp dãy số tăng dần và giảm dần (Cơ bản)

- **Mục tiêu:** Giúp học sinh nắm vững cú pháp gọi hàm sort() mặc định và tham số greater<int>().
- **Đề bài:** Nhập vào số nguyên dương n ($1 \leq n \leq 10^5$) và mảng gồm n số nguyên ($|a_i| \leq 10^9$).
    - Dòng 1: In dãy số sau khi sắp xếp **tăng dần**.
    - Dòng 2: In dãy số sau khi sắp xếp **giảm dần**.
- **Ví dụ:**

Input	**Output**
5 **4 -2 7 0 3**	**-2 0 3 4 7** **7 4 3 0 -2**

Bài 2: Tìm phần tử lớn thứ nhì khác biệt (Thông hiểu)

- **Mục tiêu:** Vận dụng sắp xếp để đưa các phần tử lớn về cuối dãy, kết hợp duyệt ngược để xử lý trường hợp các phần tử lớn nhất bị trùng lặp.
- **Đề bài:** Cho dãy gồm n số nguyên ($2 \leq n \leq 10^5$). Hãy tìm giá trị **lớn thứ nhì** trong dãy (phải nhỏ hơn giá trị lớn nhất). Nếu tất cả các phần tử trong dãy đều bằng nhau, in ra NOT FOUND.
- **Ví dụ:**

Input 1	**Output 1**
5 **7 2 7 5 3**	**5**
Input 2	**Output 2**
3 **4 4 4**	NOT FOUND

Bài 3: Khoảng cách nhỏ nhất giữa 2 phần tử bất kỳ (Vận dụng thấp)

- **Mục tiêu:** Thấy được sức mạnh vượt trội của sort(). Thay vì so sánh tất cả các cặp tốn $\mathcal{O}(n^2)$, sắp xếp giúp đưa 2 phần tử gần nhau về sát cạnh nhau, giảm về $\mathcal{O}(n \log n)$.
- **Đề bài:** Cho mảng n số nguyên ($2 \leq n \leq 10^5$, $|a_i| \leq 10^9$). Hãy tìm giá trị nhỏ nhất của $|a_i - a_j|$ với mọi cặp $i \neq j$.
- **Ví dụ:**

Input	Output
5 **1 9 15 4 20**	**3** ***(Khoảng cách nhỏ nhất là giữa 1 và 4:*** $|\mathbf{4-1}| = \mathbf{3}$***)***

Bài 4: Tách và sắp xếp chẵn lẻ riêng biệt (Vận dụng)

- **Mục tiêu:** Kỹ thuật chia luồng dữ liệu vào 2 mảng riêng biệt, áp dụng sắp xếp độc lập và ghép nối lại kết quả.
- **Đề bài:** Cho mảng gồm n số nguyên dương ($1 \leq n \leq 10^5$). Hãy sắp xếp lại mảng sao cho:
    - Các số chẵn nằm ở phía trước và được sắp xếp tăng dần.
    - Các số lẻ nằm ở phía sau và được sắp xếp giảm dần.
- **Ví dụ:**

Input	Output
6 **5 2 8 3 1 6**	**2 6 8 5 3 1**

Bài 5: Tìm số lượng phần tử khác nhau trong mảng lớn

- **Mục tiêu:**
    - Giúp học sinh thấy được ứng dụng thực tế của việc sắp xếp mảng: đưa các số giống nhau về nằm liền kề nhau để kiểm tra và đếm cực kỳ dễ dàng.
    - Giải quyết được bài toán khi giá trị phần tử rất lớn ($a_i \leq 10^9$) mà mảng đếm phân phối (Frequency Array) không làm được vì không thể tạo mảng kích thước 10^9.
- **Đề bài:** Cho mảng gồm n số nguyên ($1 \leq n \leq 10^5$, các phần tử $|a_i| \leq 10^9$). Hãy đếm xem trong mảng có tất cả bao nhiêu số khác nhau đôi một.
- **Ví dụ:**

Input	Output
6 15 3 15 1000000000 3 15	3 *(Các số khác biệt là: 3, 15, 1000000000)*

Phân tích & Hướng dẫn cho học sinh:

1. Ban đầu các số nằm lộn xộn, muốn biết số nào đã gặp rồi thì rất khó.

2. Dùng sort(a, a + n); để xếp mảng tăng dần. Lúc này, toàn bộ các phần tử có cùng giá trị sẽ dồn về đứng sát cạnh nhau.
    - Ví dụ sau khi xếp: 3, 3, 15, 15, 15, 1000000000
3. Ta chỉ cần duyệt qua mảng từ vị trí thứ 1 (i = 1 đến n - 1), nếu phần tử hiện tại khác phần tử đứng ngay trước nó (a[i] != a[i - 1]) thì đó chính là một giá trị mới xuất hiện!

MẢNG ĐỘNG (VECTOR)

1. Đặt vấn đề: Điểm yếu của Mảng 1 chiều truyền thống (Mảng tĩnh) là gì?

Khi học mảng 1 chiều tĩnh (int a[100];), các em gặp 2 khó khăn lớn:

1. **Phải đoán trước kích thước:** Luôn phải khai báo một con số cố định (ví dụ 100 hay 100005). Khai báo nhỏ quá thì tràn mảng (bị lỗi), khai báo to quá thì lãng phí bộ nhớ.
2. **Không co giãn được:** Đã khai báo 10 ô thì đúng 10 ô, không thể tự nở ra khi cần thêm phần tử hay co lại khi bớt đi.

Hình ảnh so sánh trực quan:

- **Mảng tĩnh (array):** Giống như một **khay đá bằng nhựa cứng cố định 10 ngăn**. Dù chỉ để 2 viên đá thì khay vẫn chiếm nguyên diện tích 10 ngăn; còn muốn để viên thứ 11 thì hoàn toàn không nhét thêm được.
- **Vector (std::vector):** Giống như một **chiếc túi vải co giãn thông minh**. Ban đầu túi rỗng, bỏ 1 đồ vật vào thì túi nở ra vừa đủ 1 chỗ, bỏ thêm 10 đồ vật nữa túi lại tự động dãn ra đúng 11 chỗ. Ta không bao giờ sợ thiếu chỗ hay phải đoán trước kích thước.



2. Cú pháp khai báo và Khởi tạo

Để dùng được vector, các em cần mở thư viện:

#include <vector>

Tuy nhiên thư viện này đã tích hợp sẵn trong thư viện: **#include<bits/stdc++.h>**

Cú pháp khai báo:

vector<kieu_du_lieu> ten_vector;

VD:

Ba cách khai báo thường dùng nhất:

- vector<int> a; → Tạo một vector rỗng hoàn toàn (kích thước ban đầu bằng 0).
- vector<int> a(n); → Tạo vector có sẵn đúng n phần tử (tất cả tự động mang giá trị 0).
- vector<int> a(n, 5); → Tạo vector có n phần tử, mỗi phần tử đều có giá trị ban đầu là 5.

3. Bộ 4 thao tác "vàng" cần nhớ của Vector

Thao tác	Cú pháp	Ý nghĩa
Thêm phần tử vào đuôi	a.push_back(x);	Đẩy giá trị x vào cuối vector (túi tự nở thêm 1 ô).
Lấy số lượng phần tử hiện tại	a.size()	Cho biết hiện tại vector đang chứa bao nhiêu số.
Truy cập phần tử	a[i]	Giống hệt mảng 1 chiều, chỉ mục đánh số từ 0 đến a.size() - 1.
Xóa sạch vector	a.clear();	Xóa hết toàn bộ phần tử, đưa kích thước về 0.

4. So sánh code: Nhập xuất Mảng tĩnh vs Vector

Giả sử đề bài: Nhập vào số n, sau đó nhập n số nguyên và in ra màn hình.

Cách 1: Tạo vector rỗng rồi nạp dần bằng push_back() (Khuyên dùng khi chưa biết trước số lượng dữ liệu)

```cpp
#include<bits/stdc++.h>
using namespace std;
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;

    vector<int> a; // Ban đầu túi rỗng

    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        a.push_back(x); // Nạp x vào cuối vector, vector tự nó sẽ to ra
    }

    // In các phần tử (dùng a.size() để lấy độ dài vector)
    for (int i = 0; i < a.size(); i++) {
        cout << a[i] << " ";
    }

    return 0;
}
```

Cách 2: Khai báo sẵn n ô rồi dùng như mảng thông thường

```cpp
#include<bits/stdc++.h>
using namespace std;
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;

    vector<int> a(n); // Tao luon dung n o nho

    for (int i = 0; i < n; i++) {
        cin >> a[i]; // Nhap truc tiep vao o thu i nhu mang binh thuong
    }

    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }
    cout << "\n";

    return 0;
}
```

5. Sắp xếp Vector với hàm sort()

Với mảng tĩnh, ta viết: **sort(a, a + n);**

Với vector, cú pháp thậm chí còn tự nhiên hơn vì dùng điểm đầu (**a.begin()**) và điểm cuối (**a.end()**):

- **Sắp xếp tăng dần:**

```cpp
sort(a.begin(), a.end());
```

- **Sắp xếp giảm dần:**

```cpp
sort(a.begin(), a.end(), greater<int>());
```

6. Hai lỗi sai học sinh lớp 9 rất hay mắc

a. **Nhầm lẫn giữa push_back() và gán chỉ số a[i]:**

```
vector<int> a; // a dang rong (size = 0)
cin >> a[0];   // SAI NGHIEM TRONG! O so 0 chua he ton tai -> Bao loi Crash ngay.
```

Nguyên tắc:

- **Nếu khai báo rỗng vector<int> a;, muốn nạp dữ liệu bắt buộc phải dùng a.push_back(x);.**
- **Chỉ dùng cin >> a[i]; khi đã khai báo kích thước từ đầu: vector<int> a(n);.**

b. **Quên mất chỉ số bắt đầu từ 0:**

- **Phần tử đầu tiên là a[0].**
- **Phần tử cuối cùng là a[a.size() - 1]. Truy cập a[a.size()] sẽ bị tràn giới hạn.**

BÀI TẬP VẬN DỤNG:

Bài 1: Nạp động và in dãy số kèm kích thước thực tế

- **Mục tiêu:** Thực hành tạo vector rỗng, nạp phần tử bằng push_back() và sử dụng phương thức size().
- **Đề bài:** Nhập vào số nguyên dương n ($\mathbf{1 \leq n \leq 10^5}$) và lần lượt n số nguyên. Hãy nạp toàn bộ các số vào một vector, sau đó in ra kích thước của vector trên dòng 1 và in toàn bộ dãy số theo đúng thứ tự trên dòng 2.
- **Ví dụ:**
  - **Input:**

```
4
12 5 7 9
```

  - Output:

```
Kich thuoc: 4
12 5 7 9
```

CODE MẪU:

```cpp
#include<bits/stdc++.h>
using namespace std;
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<int> a; // Khoi tao vector rong

    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        a.push_back(x); // Nap tung phan tu vao cuoi vector
    }

    // In so luong phan tu thuc te
    cout << "Kich thuoc: " << a.size() << "\n";

    // Duyet qua cac phan tu bang chi so index
    for (int i = 0; i < a.size(); i++) {
        cout << a[i] << " ";
    }
    cout << "\n";

    return 0;
}
```

Bài 2: Đọc dữ liệu không biết trước số lượng N (Đọc đến hết luồng vào)

- **Mục tiêu:** Thấy được ưu thế vượt trội của vector so với mảng tĩnh khi đề bài không cho trước số lượng phần tử n.
- **Đề bài:** Nhập vào một dãy số nguyên không biết trước số lượng cho đến khi hết dữ liệu (End of File - EOF). Hãy in ra tổng các số và dãy số theo thứ tự ngược lại từ cuối về đầu.
- **Ví dụ:**
  - **Input:**

```
4 8 1 5
```

  - **Output:**

```
Tong: 18
5 1 8 4
```

CODE MẪU:

```cpp
#include<bits/stdc++.h>
using namespace std;
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    vector<int> a;
    int x;

    // Vong lap doc lien tuc cho den khi het du lieu (EOF hoac nhan Ctrl+Z / Ctrl+D tren terminal)
    while (cin >> x) {
        a.push_back(x);
    }

    long long tong = 0;
    for (int i = 0; i < a.size(); i++) {
        tong += a[i];
    }
    cout << "Tong: " << tong << "\n";

    // Duyet nguoc tu chi so cuoi ve dau
    for (int i = a.size() - 1; i >= 0; i--) {
        cout << a[i] << " ";
    }
    cout << "\n";

    return 0;
}
```

KẾT QUẢ:

```
4 8 1 5
^Z
Tong: 18
5 1 8 4
```

Bài 3: Phân loại chẵn và lẻ vào hai vector riêng biệt

- **Mục tiêu:** Sử dụng vector để lọc dữ liệu động mà không cần ước lượng trước kích thước của mảng chẵn hay mảng lẻ.
- **Đề bài:** Cho dãy gồm n số nguyên ($1 \leq n \leq 10^5$). Hãy phân loại các số chẵn vào một vector và các số lẻ vào một vector khác. In ra số lượng và danh sách các phần tử của từng nhóm.
- **Ví dụ:**

- **Input:**

```
6
3 8 2 7 9 4
```

- **Output:**

```
Chan (3): 8 2 4
Le (3): 3 7 9
```

Bài 4: Lọc các phần tử lớn hơn trung bình cộng

- **Mục tiêu:** Rèn luyện quy trình 2 bước: duyệt vector để tính đại lượng đặc trưng (TBC), sau đó nạp các phần tử thỏa mãn điều kiện vào một vector kết quả mới.
- **Đề bài:** Nhập vào n số nguyên ($1 \leq n \leq 10^5$). Hãy tìm giá trị trung bình cộng (TBC) của cả dãy, sau đó lọc tất cả các phần tử có giá trị **lớn hơn TBC** vào một vector mới và in ra dãy số mới này.
- **Ví dụ:**

Input:

```
5
2 8 4 10 6
```

Output:

```
TBC: 6
Ket qua: 8 10
```

Bài 5: Xóa các phần tử trùng lặp sau khi sắp xếp

- **Mục tiêu:** Kết hợp hàm std::sort() với vector (begin(), end()) và kỹ thuật tạo vector mới chứa các phần tử duy nhất (Unique Elements).
- **Đề bài:** Cho mảng n số nguyên ($1 \leq n \leq 10^5$, $|a_i| \leq 10^9$). Hãy sắp xếp mảng theo thứ tự tăng dần, sau đó tạo một vector mới chỉ chứa các phần tử khác nhau đôi một (mỗi giá trị trùng chỉ giữ lại 1 lần) và in ra màn hình.
- **Ví dụ:**
- **Input:**

```
7
5 2 8 2 5 1 8
```

- **Output: 1 2 5 8**